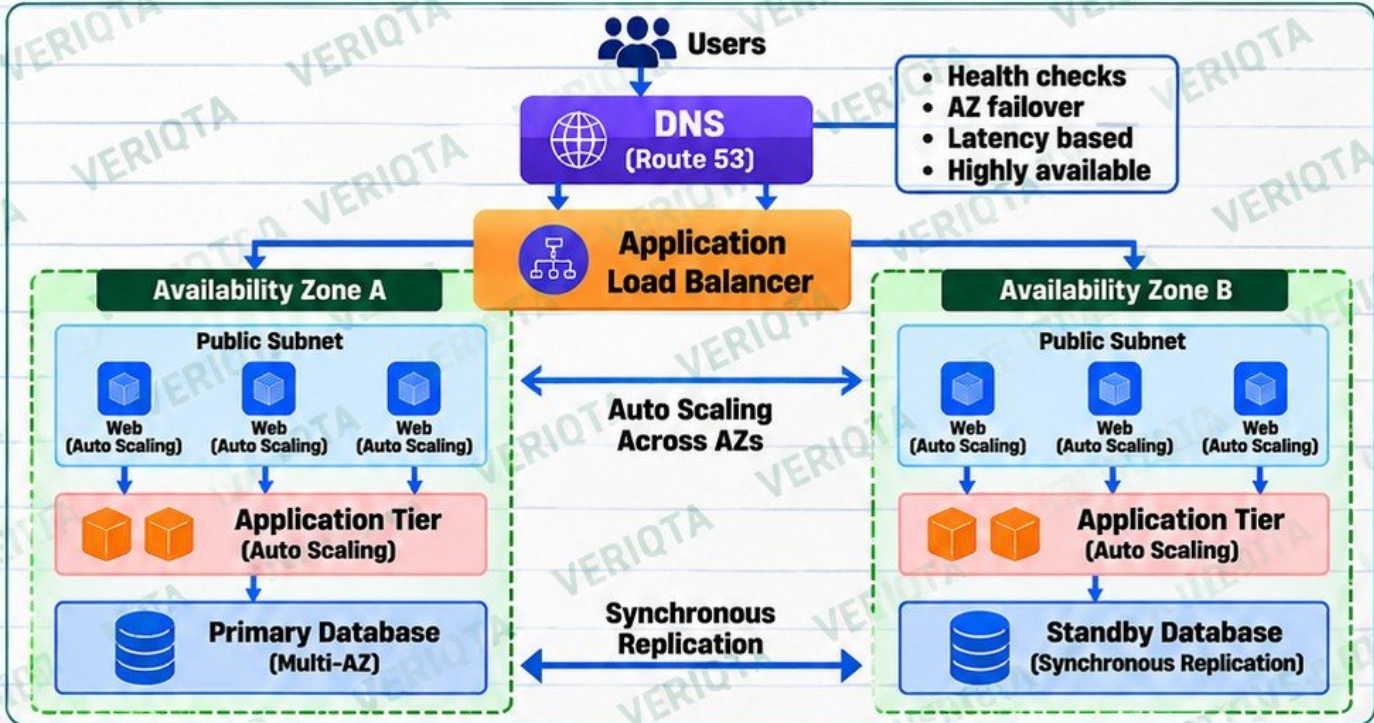


20 ADVANCED INFRASTRUCTURE ENGINEERING PROJECTS



BUILD A MULTI-AZ RESILIENT INFRASTRUCTURE

Failure domains, redundancy, capacity, and AZ failover.



WHY MULTI-AZ?

- ✓ Eliminates single point of failure
- ✓ Survives AZ outages
- ✓ Improves availability and reliability
- ✓ Handles infrastructure and network failures
- ✓ Provides automatic failover for critical services
- ✓ Supports zero or minimal downtime

KEY COMPONENTS

- DNS (Route 53) – health checks and failover
- Application Load Balancer – distributes traffic across AZs
- Auto Scaling Groups – replace unhealthy instances
- Application tier – stateless and horizontally scaled
- Multi-AZ Database – synchronous or async replication
- VPC – subnets across multiple AZs
- Monitoring – metrics, logs, alarms
- IAM and security groups – least privilege access

AZ FAILURE SCENARIO

- ✗ AZ-A becomes unavailable (e.g., power, network, or hardware)
- ✗ Instances in AZ-A fail health checks
- ✗ ALB stops sending traffic to unhealthy targets
- ✗ Route 53 shifts traffic to healthy AZ (AZ-B)
- ✗ Auto Scaling replaces instances when AZ recovers
- ✗ Database continues with standby in AZ-B

FAILOVER FLOW

- 1 Health checks detect unhealthy resources
- 2 Targets in AZ-A marked as unhealthy
- 3 ALB routes traffic to healthy targets in AZ-B
- 4 DNS fails over if needed
- 5 Application continues to serve traffic
- 6 Auto Scaling replaces instances when AZ recovers

USEFUL COMMANDS (AWS CLI)

```
aws autoscaling describe-auto-scaling-groups \
  --query "AutoScalingGroups[0].(Name:AutoScalingGroupName,AZs:AvailabilityZones)"

aws elbv2 describe-target-health --target-group-arn <TG-ARN>

aws rds describe-db-instances --query \
  "DBInstances[0].(ID:DBInstanceIdentifier,AZ:AvailabilityZone,Status:DBInstanceStatus)"

dig example.com @8.8.8.8
```

DESIGN TRADE-OFFS

- Higher cost due to duplicated resources
- More complex networking and operations
- Cross-AZ data transfer charges
- Need for stateless application design
- More components to monitor and manage

BEST PRACTICES

- ✓ Use at least two AZs for critical workloads
- ✓ Keep applications stateless
- ✓ Enable automated health checks and failover
- ✓ Test AZ failure regularly
- ✓ Monitor capacity and ensure headroom



@veriqta



VERIQTA



VERIQTA



VERIQTA



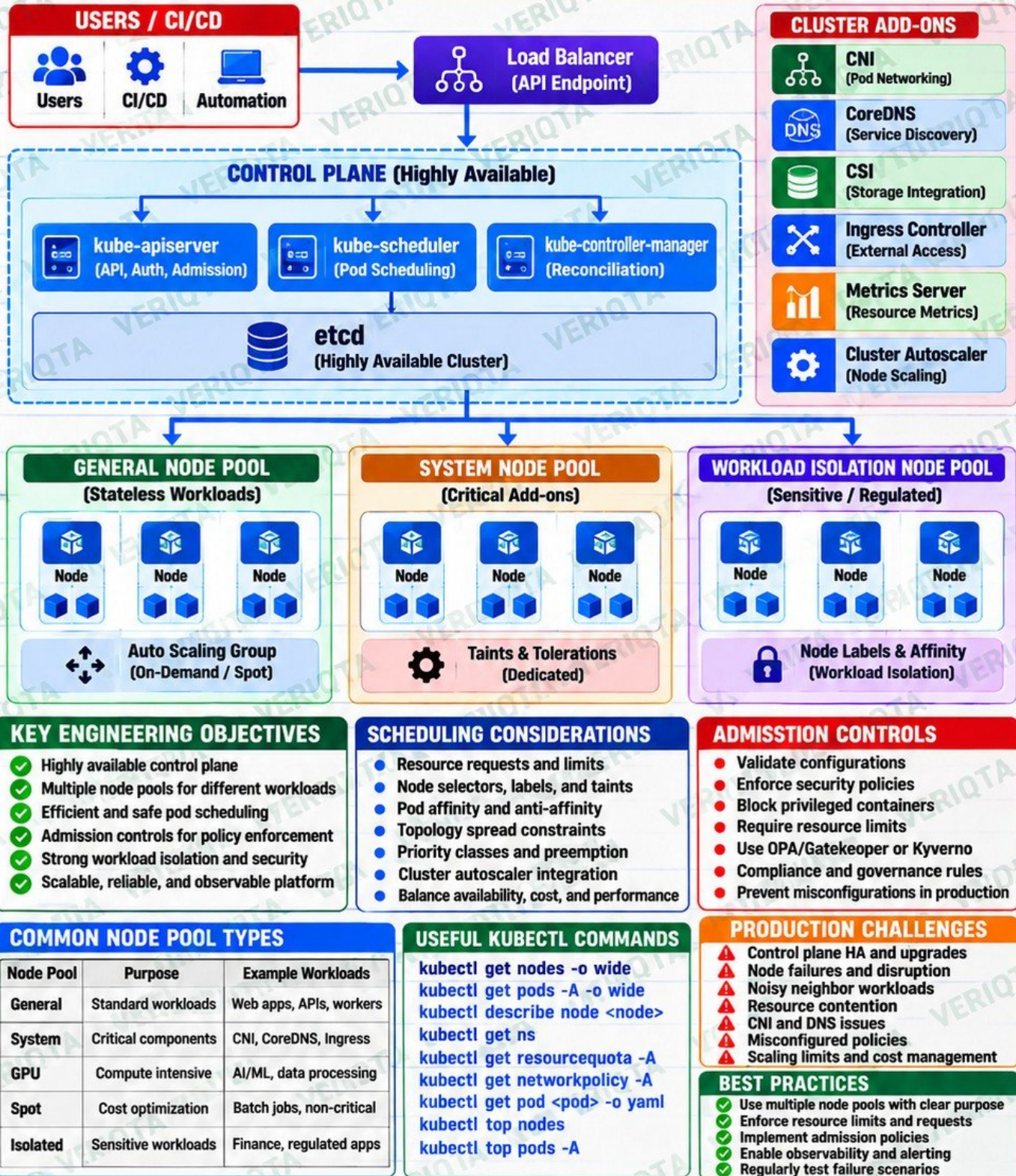
VERIQTA



@veriqta

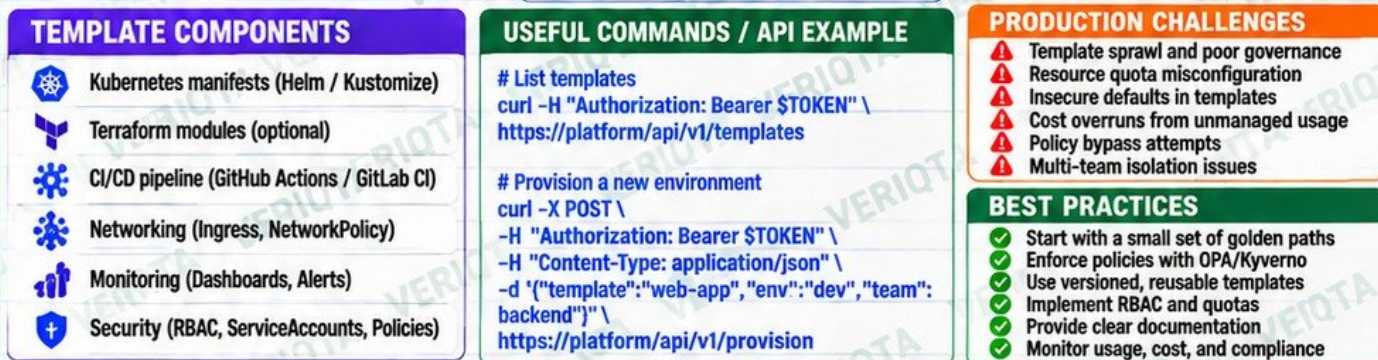
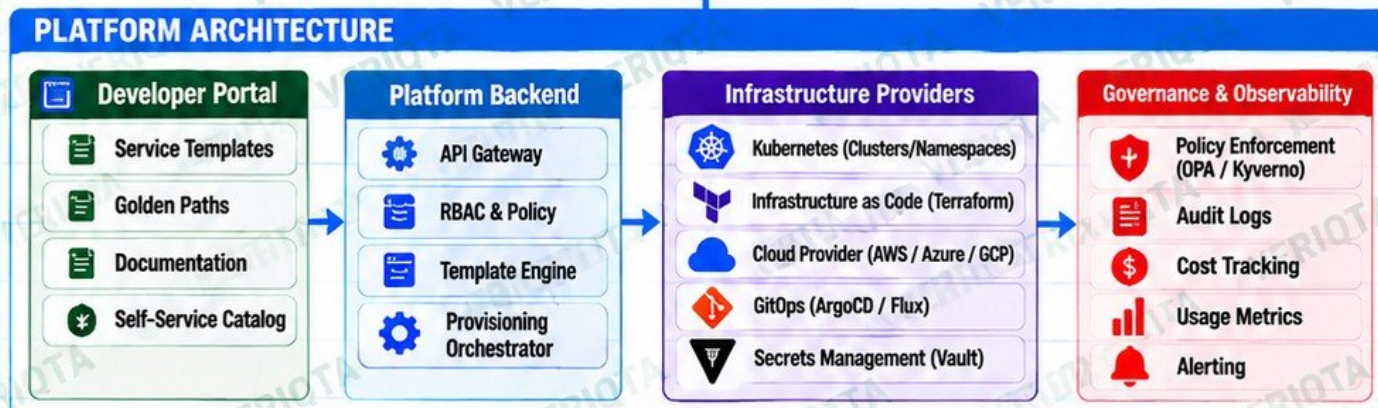
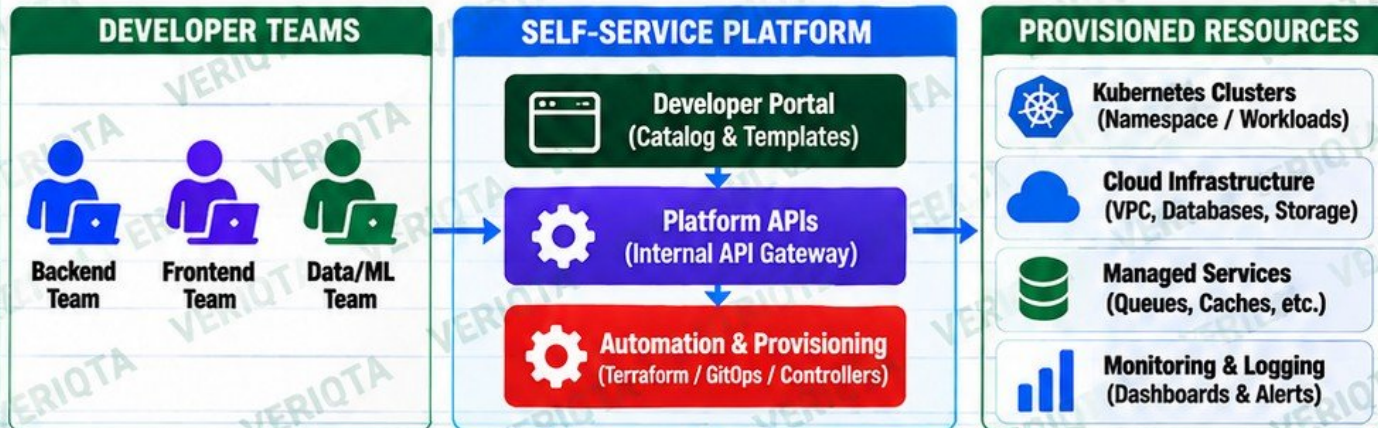
ENGINEER A PRODUCTION-GRADE KUBERNETES PLATFORM

Control plane, node pools, scheduling, admission, and workload isolation.



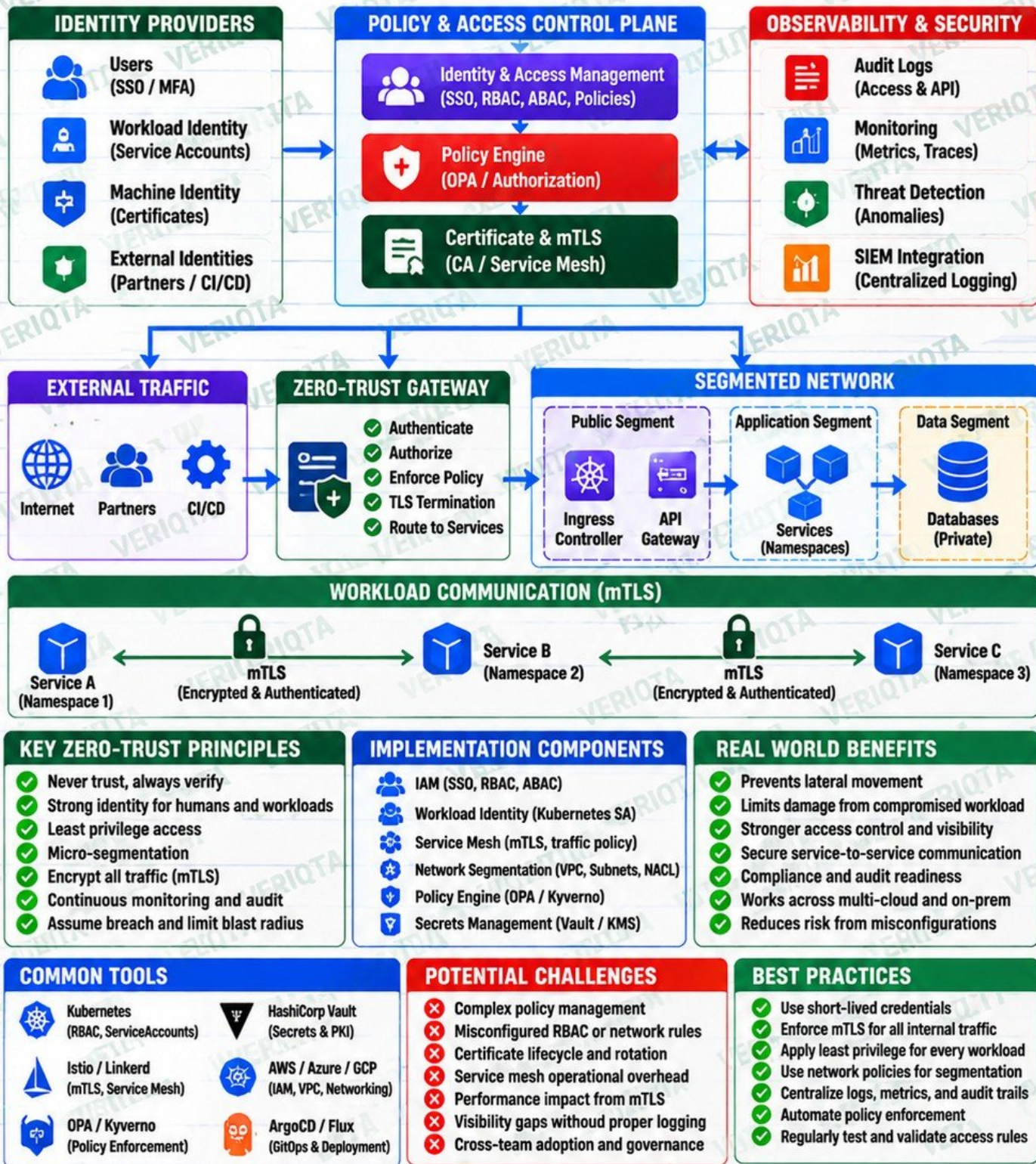
BUILD A SELF-SERVICE INTERNAL DEVELOPER PLATFORM

Platform APIs, golden paths, RBAC, and infrastructure provisioning.



DESIGN A ZERO-TRUST INFRASTRUCTURE ARCHITECTURE

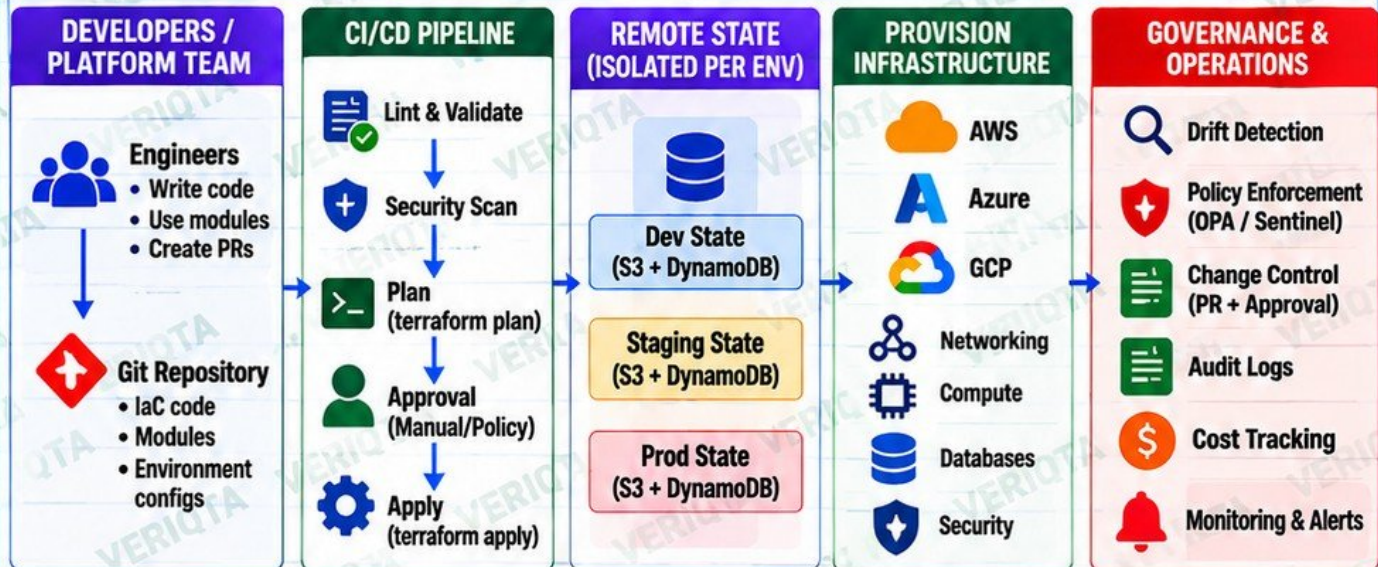
Workload identity, mTLS, segmentation, and least privilege.



ENGINEER INFRASTRUCTURE AS CODE AT ENTERPRISE SCALE

Terraform state isolation, modules, drift detection, and deployment controls.

ENTERPRISE IaC ARCHITECTURE (TERRAFORM)



TERRAFORM STATE ISOLATION

- ✓ Separate state per environment
- ✓ Use remote backend (e.g., S3 + DynamoDB)
- ✓ Enable state locking
- ✓ Restrict access with IAM
- ✓ Use workspace or separate accounts
- ✓ Encrypt state at rest and in transit
- ✓ Prevent manual state changes

EXAMPLE REMOTE BACKEND (AWS S3)

```
terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "prod/eks/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "veriqta-terraform-lock"
    encrypt = true
  }
}
```

MODULE STRUCTURE (EXAMPLE)

```
infrastructure/
├── modules/
│   ├── vpc/
│   ├── eks/
│   ├── rds/
│   └── iam/
├── envs/
│   ├── dev/
│   ├── staging/
│   └── prod/
├── main.tf
├── variables.tf
├── outputs.tf
└── versions.tf
```

DRIFT DETECTION

- Detects manual changes in the cloud
- Use terraform plan regularly
- Automate drift detection in CI/CD
- Alert on unexpected changes
- Investigate and decide to fix or ignore
- Use read-only accounts for audit
- Track drift history and trends

DEPLOYMENT CONTROLS

- Use pull requests for all changes
- Require plan review and approval
- Enforce policies with OPA/Sentinel
- Use separate workspaces or accounts
- Limit who can apply to production
- Use automated pipelines with audit logs
- Implement rollback strategy

KEY BENEFITS

- ✓ Scalable and consistent infrastructure
- ✓ Environment isolation and safety
- ✓ Faster and more reliable deployments
- ✓ Reduced manual errors and configuration drift
- ✓ Better security and compliance
- ✓ Auditable and traceable changes
- ✓ Supports multi-team collaboration

COMMON CHALLENGES

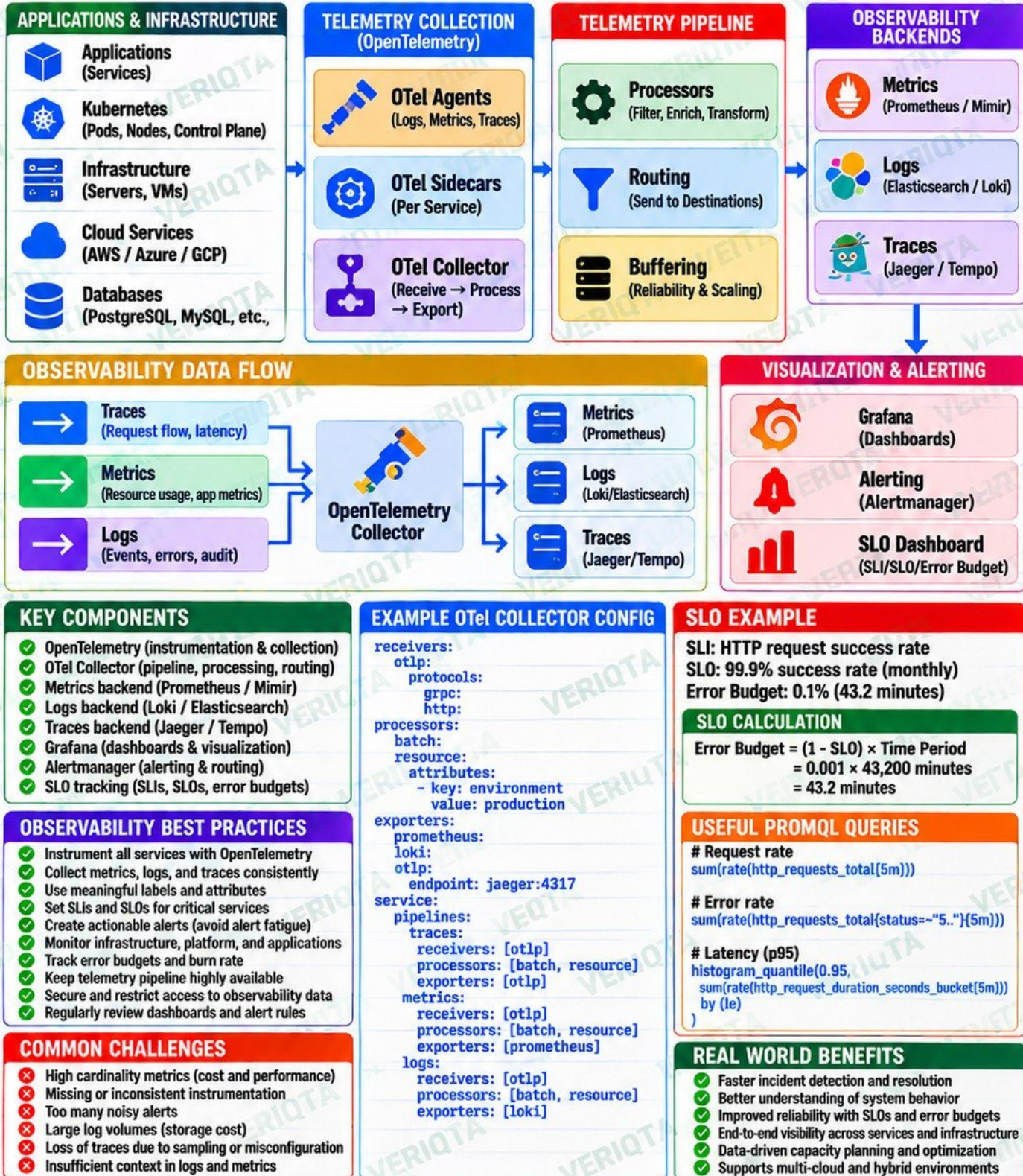
- ✗ State file corruption or lock issues
- ✗ Misconfigured IAM permissions
- ✗ Unintended resource destruction
- ✗ Module versioning and dependency drift
- ✗ Policy bypass or inconsistent workflows
- ✗ Managing multi-environment complexity
- ✗ Handling large-scale state and performance

BEST PRACTICES

- ✓ Use reusable and versioned modules
- ✓ Keep state isolated and secure
- ✓ Automate plan, policy checks, and drift detection
- ✓ Use least privilege IAM roles
- ✓ Tag all resources with metadata
- ✓ Implement strong CI/CD controls
- ✓ Regularly review and refactor modules

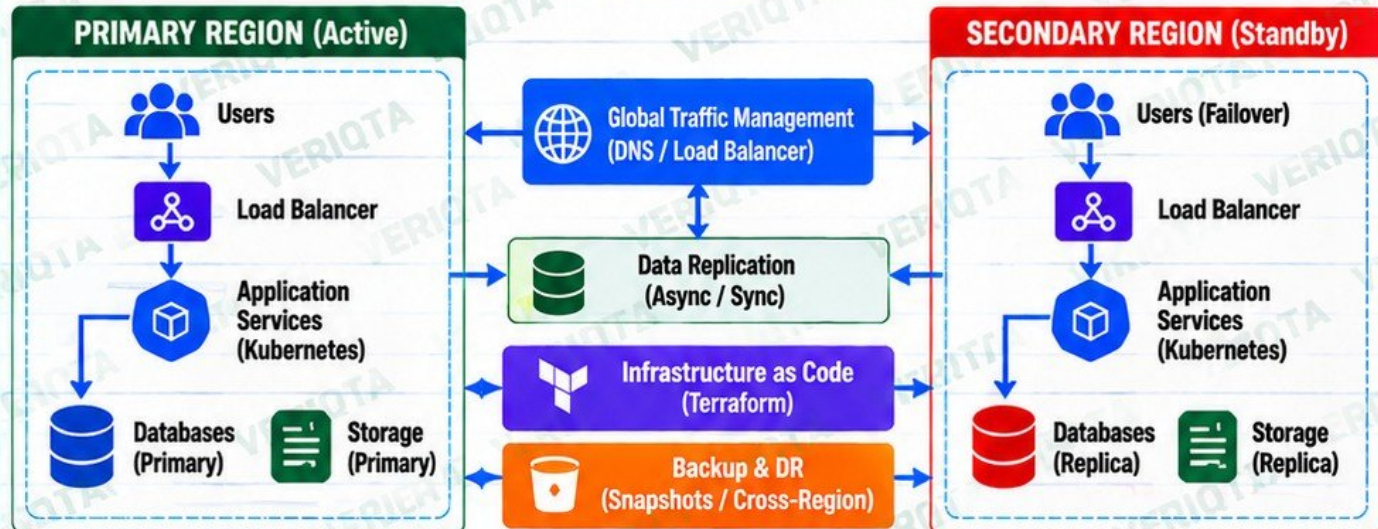
BUILD A PRODUCTION OBSERVABILITY PLATFORM

OpenTelemetry, metrics, logs, traces, SLOs, and telemetry pipelines.



ENGINEER A MULTI-REGION DISASTER RECOVERY SYSTEM

RTO, RPO, replication, recovery orchestration, and failover validation.



DISASTER RECOVERY PROCESS



RTO vs RPO



KEY COMPONENTS

- ✓ Multi-region architecture
- ✓ Global DNS / traffic management
- ✓ Application replication (stateless/stateful)
- ✓ Database replication (cross-region)
- ✓ Infrastructure as code (Terraform)
- ✓ Backup and snapshot strategy
- ✓ Automated failover and recovery
- ✓ Monitoring and alerting
- ✓ Regular DR testing and validation

REPLICATION STRATEGIES

- Databases** (Sync / Async Replication)
- Object Storage** (Cross-Region Replication)
- File Systems** (Distributed / Replicated)
- Container Images** (Registry Replication)
- Infrastructure** (Terraform State + Modules)

FAILOVER CONSIDERATIONS

- Automated vs manual failover
- DNS TTL and propagation
- Data consistency and integrity
- Application dependencies
- Cross-region networking
- IAM and access controls
- Stateful service recovery
- Testing without impacting production
- Rollback strategy

EXAMPLE TOOLS & SERVICES



DISASTER RECOVERY TESTING

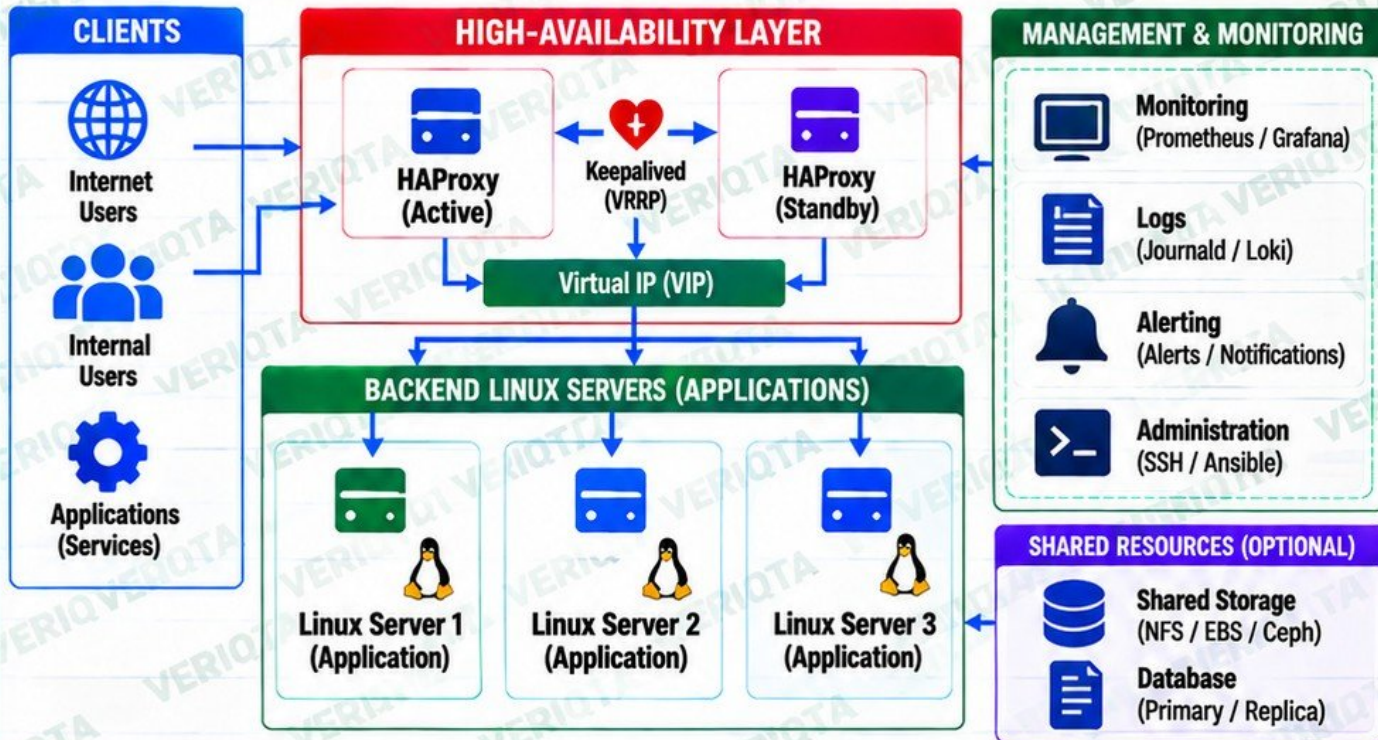
- Tabletop exercises
- Automated failover tests
- Data recovery validation
- Application health checks
- Performance verification
- Regular scheduled DR drills
- Document results and gaps

BEST PRACTICES

- ✓ Define clear RTO and RPO for each service
- ✓ Use infrastructure as code
- ✓ Automate failover and recovery processes
- ✓ Regularly test DR in a non-production setup
- ✓ Monitor replication health and lag
- ✓ Keep runbooks updated
- ✓ Validate data integrity after failover
- ✓ Implement multi-region observability
- ✓ Optimize cost while meeting DR requirements

BUILD A HIGH-AVAILABILITY LINUX SERVICE PLATFORM

systemd, HAProxy, Keepalived, kernel tuning, and process recovery.



KEY COMPONENTS

-  **systemd**
(Service management & auto-restart)
-  **HAProxy**
(Load balancing & health checks)
-  **Keepalived**
(Virtual IP & failover with VRRP)
-  **Kernel Tuning**
(Network, performance, stability)
-  **Process Recovery**
(Auto-restart, watchdog, monitoring)

EXAMPLE systemd SERVICE

```
[Unit]
Description=My Web Service
After=network.target

[Service]
ExecStart=/usr/local/bin/myapp
Restart=always
RestartSec=5
User=webuser
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

EXAMPLE HAProxy CONFIG

```
frontend http_front
    bind *:80
    default_backend web_servers

backend web_servers
    option httpchk GET /health
    server web1 10.0.1.11:80 check
    server web2 10.0.1.12:80 check
    server web3 10.0.1.13:80 check
```

FAILOVER PROCESS

- 1 Node or service failure detected
- 2 Keepalived triggers VIP failover
- 3 HAProxy routes traffic to healthy servers
- 4 systemd restarts failed services
- 5 Monitoring and alerts notify the team
- 6 Service is restored with minimal downtime

KERNEL TUNING (EXAMPLE)

```
# Network performance
net.core.somaxconn = 65536
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_tw_reuse = 1

# File descriptors
fs.file-max = 2097152

# General performance
vm.swappiness = 10
```

BEST PRACTICES

- ✓ Use systemd with restart policies
- ✓ Implement HAProxy health checks
- ✓ Use Keepalived for automatic failover
- ✓ Tune kernel parameters for performance
- ✓ Monitor services, logs, and system health
- ✓ Set up alerting and notifications
- ✓ Test failover and recovery regularly
- ✓ Document runbooks and procedures



@veriqta



VERIQTA



VERIQTA



VERIQTA



VERIQTA

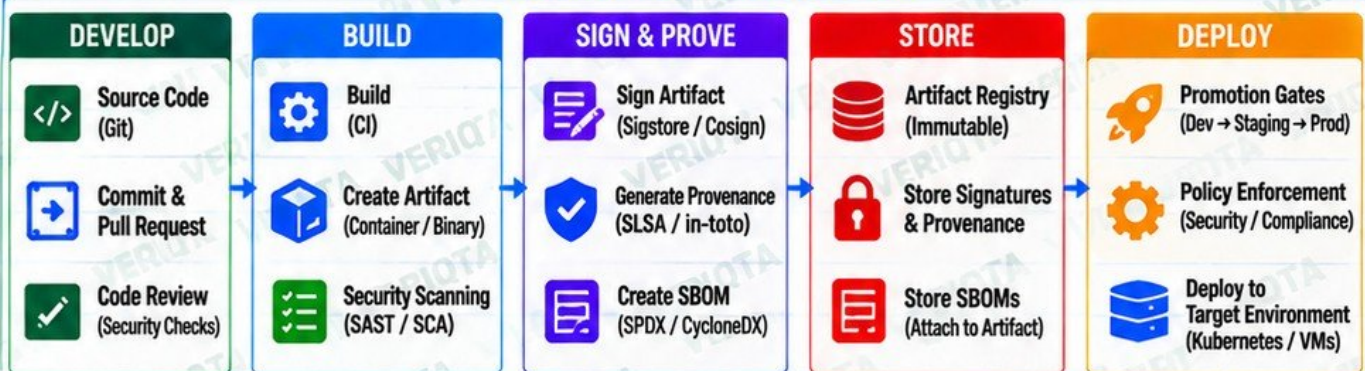


@veriqta

ENGINEER A SECURE SOFTWARE DELIVERY PLATFORM

Artifact promotion, provenance, signing, SBOMs, and deployment gates.

SECURE SOFTWARE DELIVERY PIPELINE



ARTIFACT PROMOTION FLOW



DEPLOYMENT TARGETS



KEY COMPONENTS

- Source Code Management (Git / GitHub / GitLab)
- CI/CD Platform (GitHub Actions / GitLab CI / Jenkins)
- Artifact Registry (ECR / ACR / GCR / Nexus / Artifactory)
- Signing & Provenance (Sigstore / Cosign / SLSA / in-toto)
- SBOM Generation (Syft / CycloneDX / SPDX)
- Policy & Deployment Gates (OPA / Conftest / Kyverno)
- Deployment Platform (Kubernetes / ArgoCD / Flux)

SECURITY CONTROLS

- Sign all artifacts
- Generate and store provenance
- Generate SBOMs for every build
- Scan for vulnerabilities (SAST/SCA)
- Enforce policy gates before promotion
- Use immutable artifact registries
- Verify signatures at deployment
- Track and audit artifact lineage
- Restrict access with least privilege
- Detect and block untrusted artifacts

EXAMPLE TOOLS



COMMON CHALLENGES

- Unsigned or tampered artifacts
- Missing or incomplete provenance
- No SBOMs or outdated SBOMs
- Vulnerabilities in dependencies
- Weak or missing deployment gates
- Inconsistent environments
- Lack of artifact traceability
- Overly permissive access controls

BEST PRACTICES

- Automate signing and provenance generation
- Generate SBOMs for all artifacts
- Enforce policy gates at each stage
- Use immutable and versioned registries
- Verify artifacts before deployment
- Implement least privilege access
- Continuously scan and monitor
- Maintain full audit trail and traceability
- Regularly test deployment and rollback

REAL WORLD BENEFITS

- More secure software supply chain
- Reduced risk of compromised artifacts
- Faster and safer deployments
- Compliance with industry standards
- Full visibility and traceability
- Prevents unauthorized changes
- Improved operational reliability
- Easier incident response and rollback
- Builds trust with customers and teams



@veriqta



VERIQTA



VERIQTA



VERIQTA



VERIQTA



@veriqta

BUILD A KUBERNETES FAILURE RECOVERY LAB

Node failures, OOMKilled workloads, eviction, PDBs, and reconciliation.

KUBERNETES FAILURE SCENARIOS

1. NODE FAILURE



- Node becomes NotReady
- Pods are rescheduled to healthy nodes
- Validate application recovery

2. OOMKILLED WORKLOADS



- Container exceeds memory limit
- Pod is OOMKilled and restarted
- Validate restart and stability

3. EVICTION



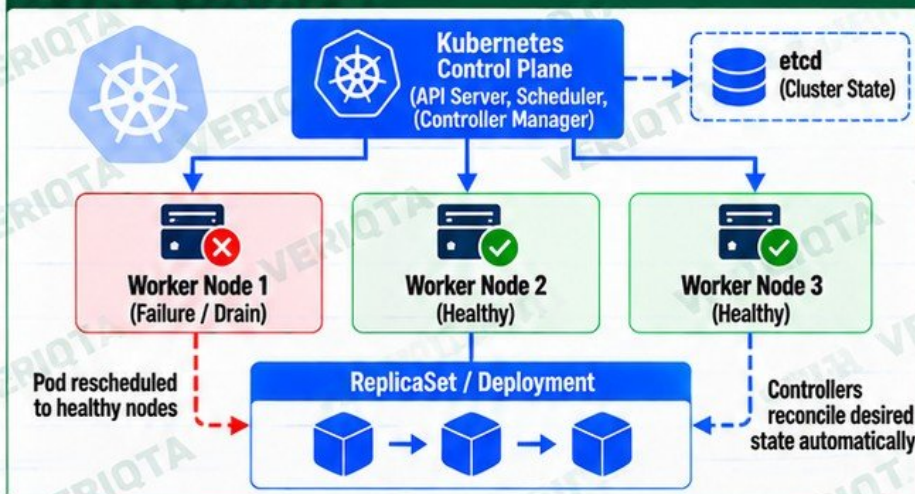
- Drain node (maintenance)
- Pods are evicted based on PDBs and priorities
- Validate graceful eviction and rescheduling

4. PDBs & RECONCILIATION



- PDB prevents too many pods from being unavailable
- Controllers reconcile and maintain desired state
- Validate application availability

ARCHITECTURE OVERVIEW



KEY KUBERNETES RESOURCES



Pods
(Application workloads)



Deployments
(Desired state)



ReplicaSets
(Pod management)



PodDisruptionBudgets
(Availability control)



Nodes
(Worker machines)



Kubelet
(Node health & pod lifecycle)



Controllers
(Reconciliation loop)

USEFUL COMMANDS

```
>_
kubectl get nodes           # Check node status
kubectl get pods -A         # Check all pods
kubectl describe pod <pod>  # Pod details
kubectl logs <pod>          # View logs
kubectl drain <node>        # Evict pods from node
kubectl uncordon <node>     # Bring node back
kubectl delete pod <pod>    # Simulate failure
kubectl get pdb             # Check PDBs
kubectl describe pdb <pdb>  # PDB details
kubectl top <pod>           # Check resource usage
```

EXAMPLE PDB CONFIG

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web
```

EXAMPLE OOMKILL POD

```
apiVersion: v1
kind: Pod
metadata:
  name: oom-demo
spec:
  containers:
    - name: stress
      image: polinux/stress
      resources:
        limits:
          memory: "100Mi"
      command: ["stress", "--vm", "1",
        "--vm-bytes", "500M", "--timeout",
        "600"]
```

COMMON CHALLENGES

- Nodes going NotReady or unreachable
- Pods OOMKilled due to memory limits
- Unexpected pod evictions
- PDB blocking voluntary disruptions
- Application downtime during maintenance
- Misconfigured resource requests and limits

BEST PRACTICES

- Set proper resource requests and limits
- Use PDBs for critical applications
- Implement liveness and readiness probes
- Distribute pods across multiple nodes
- Use node taints and tolerations correctly
- Monitor and alert on node and pod failures
- Regularly test failure scenarios
- Let controllers handle reconciliation

REAL WORLD BENEFITS

- Applications stay available during failures
- Automatic recovery and self-healing
- Controlled and safe node maintenance
- Better resource management
- Improved resilience and reliability
- Validates production readiness
- Builds confidence in Kubernetes operations
- Reduces downtime and business impact



@veriqta



VERIQTA



VERIQTA



VERIQTA



VERIQTA



@veriqta

DESIGN A PRODUCTION NETWORKING FAILURE LAB

DNS, routing, NAT, MTU, packet loss, TLS, and connection exhaustion.

DNS FAILURES



- DNS resolution failures
- Slow DNS responses
- Incorrect records (wrong IP)
- DNS cache issues
- Split-horizon DNS

ROUTING ISSUES



- Missing routes
- Incorrect routing tables
- BGP/OSPF failures
- Asymmetric routing
- Blackhole routes

NAT PROBLEMS



- NAT port exhaustion
- Incorrect NAT rules
- SNAT/DNAT issues
- Hairpin NAT problems
- Conntrack table full

MTU ISSUES



- MTU mismatch
- Packet fragmentation
- PMTUD failures
- Dropped large packets
- Application timeouts

PACKET LOSS



- High latency
- Packet drops
- Network congestion
- Queue overflows
- Intermittent connectivity

TLS PROBLEMS



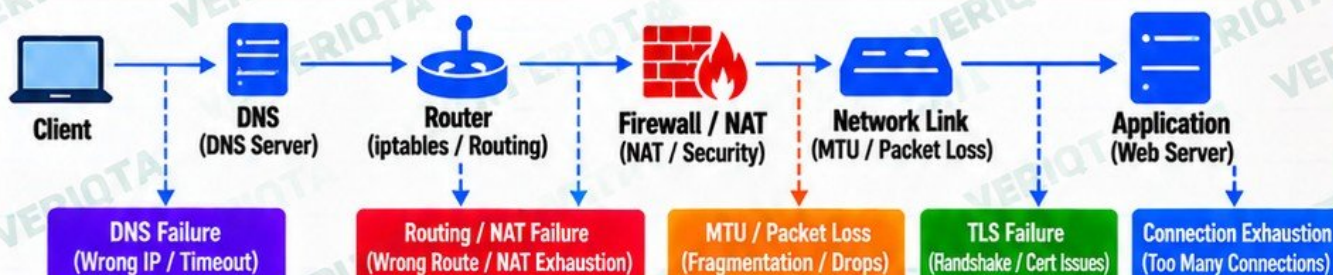
- Certificate errors
- Expired certificates
- TLS handshake failures
- Cipher mismatch
- SNI / hostname issues

CONNECTION EXHAUSTION



- Too many open connections
- Ephemeral port exhaustion
- File descriptor limits
- TIME_WAIT buildup
- Service becomes unavailable

LAB ARCHITECTURE



USEFUL COMMANDS

```

> dig example.com # DNS lookup
nslookup example.com # DNS query
ip route show # Check routes
iptables -t nat -L # Check NAT rules
tracert example.com # Check MTU path
ping -M do -s 1472 example.com # Test MTU size
tcpdump -i eth0 # Capture packets
ss -s # Connection stats
ss -tan | grep :443 # Count connections
openssl s_client -connect # Test TLS handshake
example.com:443
conntrack -S # Conntrack stats
  
```

EXAMPLE SCENARIOS

- 1 DNS returns wrong IP
- 2 Route missing to target network
- 3 NAT port exhaustion under load
- 4 MTU mismatch causes timeouts
- 5 Simulate 10% packet loss
- 6 TLS certificate expired
- 7 Open thousands of connections until service fails

MONITORING & ALERTING

- Prometheus (Metrics)
- Grafana (Visualization)
- Alertmanager (Alerts)
- Logs (Fluentd / Loki)
- Tracing (Jaeger / OpenTelemetry)

COMMON CHALLENGES

- Intermittent and hard-to-reproduce issues
- Multiple layers causing the same symptom
- DNS, routing, or NAT misconfiguration
- MTU and fragmentation problems
- Connection exhaustion under high load
- TLS and certificate issues
- Detecting the real root cause

BEST PRACTICES

- Use monitoring, logs and traces together
- Set proper timeouts and retries
- Size connection and file descriptor limits
- Validate MTU and use consistent settings
- Implement health checks and failover
- Use load testing to verify capacity
- Document runbooks for each failure type

REAL WORLD BENEFITS

- Faster troubleshooting of network issues
- More reliable and resilient services
- Improved incident response
- Better capacity planning
- Reduced downtime
- Stronger security posture
- Production-ready networking skills



@veriqta



VERIQTA



VERIQTA



VERIQTA



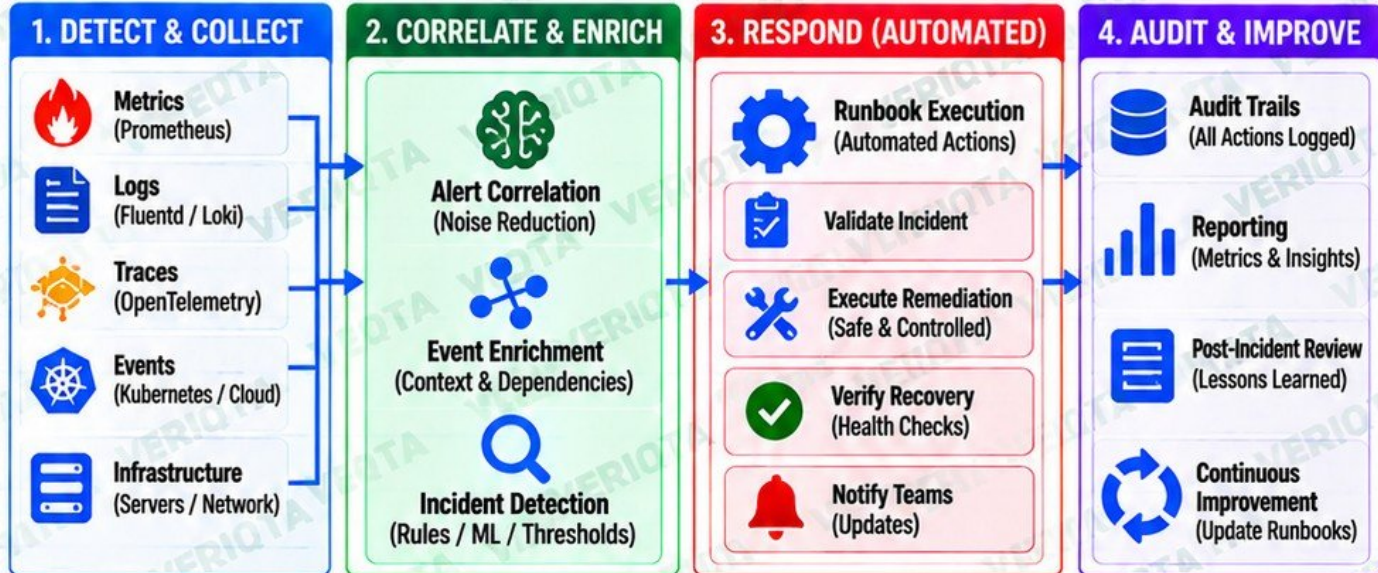
VERIQTA



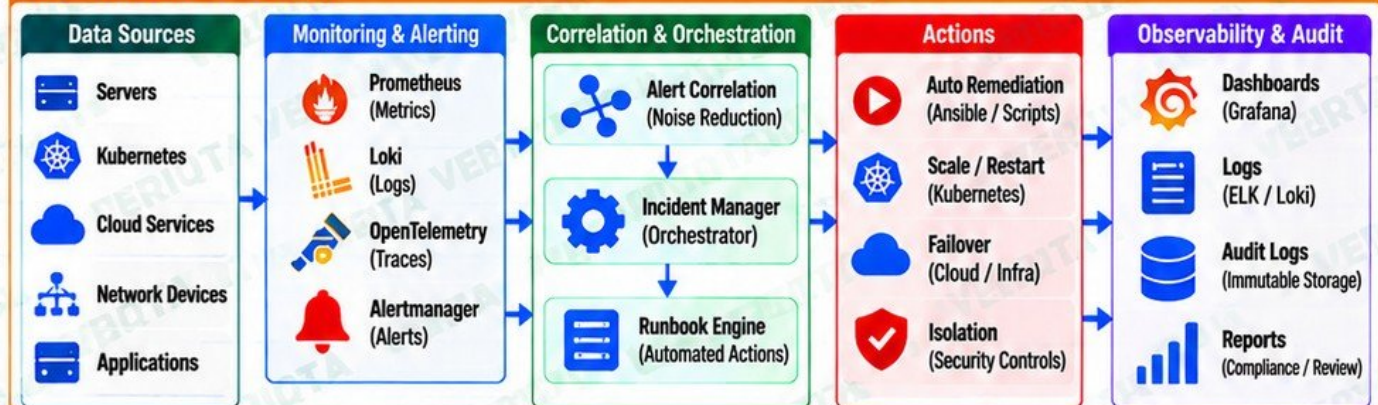
@veriqta

BUILD AN AUTOMATED INFRASTRUCTURE INCIDENT RESPONSE SYSTEM

Alert correlation, runbooks, controlled remediation, and audit trails.



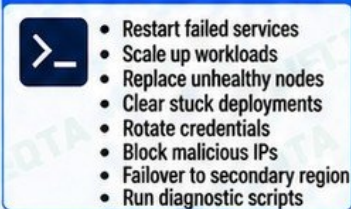
SYSTEM ARCHITECTURE



EXAMPLE RUNBOOK FLOW



EXAMPLE AUTOMATED ACTIONS



TOOLS & TECHNOLOGIES



KEY COMPONENTS

- ✓ Alert collection (metrics, logs, traces, events)
- ✓ Alert correlation and noise reduction
- ✓ Incident detection and classification
- ✓ Runbook automation and orchestration
- ✓ Controlled and safe remediation
- ✓ Verification and recovery checks
- ✓ Audit trails and immutable logs
- ✓ Reporting and continuous improvement

COMMON CHALLENGES

- ✗ Alert noise and false positives
- ✗ Missing context and dependencies
- ✗ Unsafe automated actions
- ✗ Incomplete remediation
- ✗ Lack of audit trails
- ✗ Permission and access issues
- ✗ Complex multi-cloud environments
- ✗ Runbooks not updated
- ✗ Slow incident response

BEST PRACTICES

- ✓ Use clear alerting rules and thresholds
- ✓ Reduce noise with correlation
- ✓ Automate safe and reversible actions
- ✓ Implement approval for high-risk actions
- ✓ Maintain up-to-date runbooks
- ✓ Log all actions with audit trails
- ✓ Test runbooks regularly
- ✓ Monitor the remediation process
- ✓ Perform post-incident reviews

REAL WORLD BENEFITS

- ✓ Faster incident response
- ✓ Reduced downtime
- ✓ More reliable infrastructure
- ✓ Improved service availability
- ✓ Consistent and safe remediation
- ✓ Full audit and compliance tracking
- ✓ Lower operational burden
- ✓ Better visibility and insights
- ✓ Continuous improvement

ENGINEER A SCALABLE STATEFUL WORKLOAD PLATFORM

Persistent storage, replication, consistency, backups, and recovery.

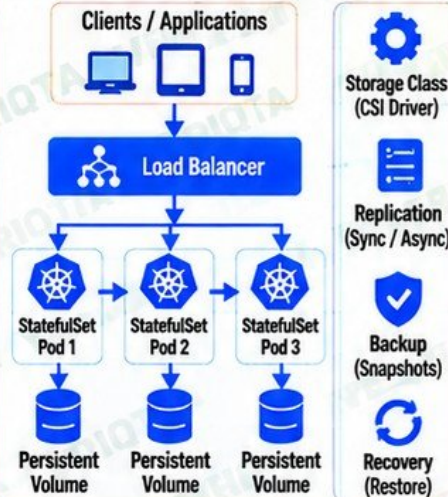
STATEFUL WORKLOADS

- Databases (PostgreSQL / MySQL)
- Distributed Databases (MongoDB / Cassandra)
- Key-Value Stores (Redis)
- Message Queues (Kafka / RabbitMQ)
- Other Stateful Apps (Elasticsearch / etcd)

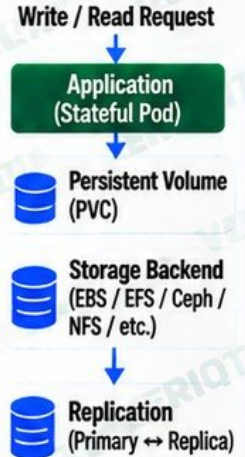
KEY REQUIREMENTS

- Persistent Storage
- Data Replication
- Data Consistency
- High Availability
- Scalability
- Automated Backups
- Point-in-Time Recovery
- Disaster Recovery
- Monitoring & Alerting

ARCHITECTURE OVERVIEW



DATA FLOW



STORAGE OPTIONS

- AWS EBS (Block Storage)
- AWS EFS (Shared File Storage)
- Ceph (Distributed Storage)
- NFS (Network File System)
- Local Storage (High Performance)

REPLICATION MODELS

Synchronous Replication



- Strong consistency
- Higher latency
- Used for critical data

Asynchronous Replication



- Better performance
- Eventual consistency
- Used for large-scale and cross-region

BACKUP AND RECOVERY

- Automated Backups (Scheduled snapshots)
- Point-in-Time Recovery (Restore to specific time)
- Cross-Region Backups (Disaster recovery)
- Backup Retention (Daily / Weekly / Monthly)
- Regular Restore Testing (Verify backups)

CONSISTENCY MODELS

- Strong Consistency (Immediate consistency)
- Eventual Consistency (Replica lag allowed)
- Quorum-Based (Majority agreement)

SCALING STRATEGIES

- Horizontal Scaling (Add more replicas)
- Vertical Scaling (Increase resources)
- Sharding / Partitioning (Distribute data)
- Read Replicas (Scale read traffic)

COMMON CHALLENGES

- Data loss due to node failure
- Inconsistent replicas
- Backup failures
- Slow recovery time
- Storage performance bottlenecks
- Split brain in distributed systems
- Misconfigured storage classes
- Lack of backup testing

BEST PRACTICES

- Use StatefulSets with proper storage classes
- Enable replication and high availability
- Choose the right consistency model
- Automate and regularly test backups
- Monitor storage, replication, and health
- Implement disaster recovery plan
- Use resource limits and requests
- Document runbooks and procedures

EXAMPLE KUBERNETES CONFIG

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db
  replicas: 3
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    volumeClaimTemplates:
      - metadata:
          name: data
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 100Gi
          storageClassName: fast-ssd
```

REAL WORLD BENEFITS

- Highly available stateful applications
- Data durability and integrity
- Scalable and resilient storage platform
- Faster recovery from failures
- Reduced downtime and data loss
- Supports business critical workloads
- Improved performance and reliability
- Ready for multi-region deployments

BUILD AN INFRASTRUCTURE CAPACITY ENGINEERING SYSTEM

Saturation, forecasting, resource contention, autoscaling, and cost.

KEY COMPONENTS



Monitoring & Metrics
(CPU, Memory, Disk, Network, I/O, Requests)



Historical Data
(Trends & Seasonality)



Forecasting Engine
(Predict Future Demand)



Autoscaling & Resource Management
(Scale Out / Scale In)



Cost Optimization
(Right Sizing & Savings)



Reporting & Alerts
(Capacity Risks & Actions)

CAPACITY ENGINEERING PIPELINE



1. COLLECT
Gather metrics, logs and usage data



2. ANALYZE
Identify current usage, saturation and trends



3. FORECAST
Predict future demand (days, weeks, months)



4. PLAN
Define capacity, scaling and cost strategies



5. EXECUTE
Implement autoscaling, rightsizing and optimizations



6. REVIEW
Continuous monitoring, adjust and improve

KEY METRICS TO TRACK



CPU Usage
(Utilization, Steal, Saturation)



Memory Usage
(Utilization, Page Faults, OOM)



Disk I/O
(Read/Write, IOPS, Latency)



Network
(Bandwidth, Packet Loss, Connections)



Application Metrics
(Requests, Latency, Error Rate)

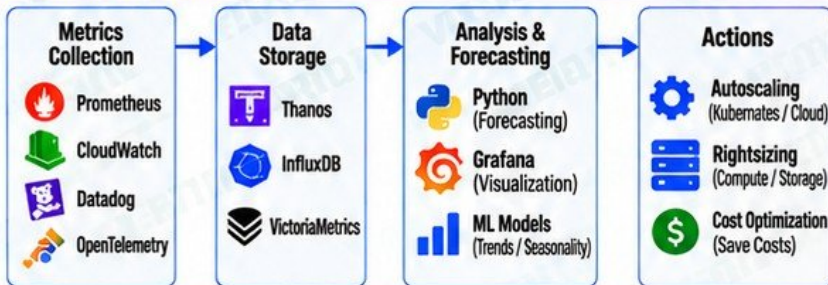


Resource Saturation
(CPU, Memory, Disk, Network)

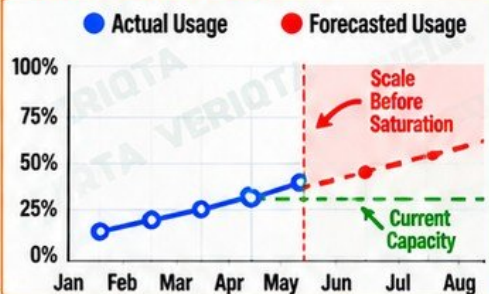


Cost Metrics
(Spend, Idle Resources, Rightsizing)

SYSTEM ARCHITECTURE



FORECASTING EXAMPLE



EXAMPLE AUTOSCALING CONFIG (KUBERNETES HPA)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

TOOLS & TECHNOLOGIES



Prometheus
(Metrics Collection)



Grafana
(Visualization)



Datadog
(Monitoring & Alerts)



Kubernetes HPA
(Autoscaling)



AWS / Azure / GCP
(Cloud Autoscaling)



InfluxDB / Thanos
(Metrics Storage)



Python
(Forecasting / Analysis)



Terraform
(Infrastructure Planning)

COMMON SCENARIOS



Traffic Spike
(Scale automatically)



Resource Contention
(CPU, Memory, I/O)



Saturation
(Near capacity limits)



Storage Growth
(Plan additional storage)



Unexpected Demand
(Promotions / Launches)



Cost Optimization
(Rightsize unused resources)

REAL WORLD BENEFITS

- ✓ Prevent service outages
- ✓ Better performance and reliability
- ✓ Accurate capacity planning
- ✓ Optimized resource usage
- ✓ Lower infrastructure costs
- ✓ Faster response to demand changes
- ✓ Data-driven decision making
- ✓ Efficient autoscaling
- ✓ Improved customer experience
- ✓ Stronger financial control

COMMON CHALLENGES

- ✗ Inaccurate metrics or missing data
- ✗ Unpredictable traffic patterns
- ✗ Resource contention (CPU, Memory, I/O)
- ✗ Over-provisioning (wasted cost)
- ✗ Under-provisioning (performance issues)
- ✗ No forecasting or poor models
- ✗ Slow scaling or misconfigured autoscaling
- ✗ Multiple environments and dependencies
- ✗ Cost spikes and budget overruns
- ✗ Lack of continuous review and tuning

BEST PRACTICES

- ✓ Monitor all key resources and application metrics
- ✓ Set realistic resource requests and limits
- ✓ Use historical data for accurate forecasting
- ✓ Implement autoscaling with proper thresholds
- ✓ Use multiple metrics for scaling decisions
- ✓ Regularly review and optimize resource usage
- ✓ Automate cost analysis and alerts
- ✓ Test scaling and failure scenarios
- ✓ Continuously refine forecasts and capacity plans



@veriqta



VERIQTA



VERIQTA



VERIQTA



VERIQTA



@veriqta

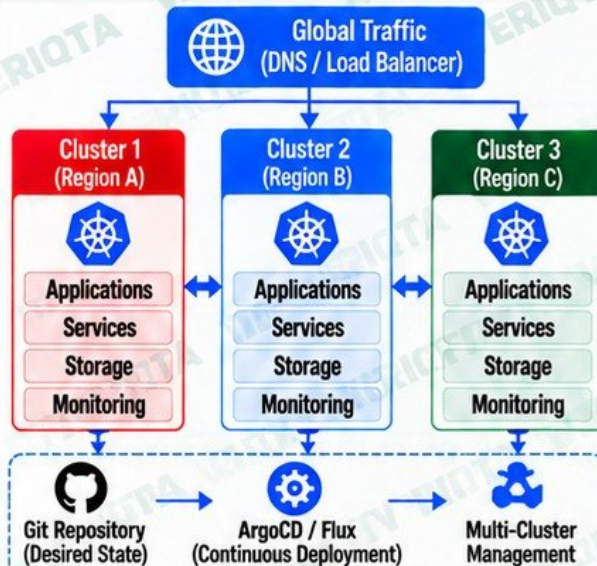
ENGINEER A MULTI-CLUSTER KUBERNETES ARCHITECTURE

Cluster boundaries, workload placement, networking, and GitOps.

WHY MULTI-CLUSTER?

-  High availability and disaster recovery
-  Scalability and resource isolation
-  Security and compliance boundaries
-  Workload flexibility and placement
-  Better resilience and cost optimization

ARCHITECTURE OVERVIEW



KEY CONCEPTS

-  Cluster Boundaries (Isolation and tenancy)
-  Workload Placement (Region, compliance, latency)
-  Networking (Multi-cluster communication)
-  GitOps (Declarative deployment)
-  Policy and Security (RBAC, Network Policies)
-  Storage and Data (Replication and backups)
-  Observability (Centralized monitoring)

CLUSTER BOUNDARIES

-  By region (multi-region)
-  By environment (dev/stage/prod)
-  By team (multi-tenancy)
-  By compliance (data residency)
-  By workload type (stateless/stateful)

WORKLOAD PLACEMENT STRATEGIES

-  Latency-based placement (closest to users)
-  Resource-based placement (CPU, memory, GPU)
-  Regulatory requirements (data residency)
-  Failover and disaster recovery (active/active or active/passive)
-  Cost-based placement (optimize cloud spend)

MULTI-CLUSTER NETWORKING

-  Service discovery across clusters
-  Cluster-to-cluster communication (service mesh / CNI)
-  Secure networking (mTLS, VPN, PrivateLink)
-  Ingress and egress strategies
-  Network policies and segmentation
-  Latency and bandwidth optimization

GITOPS MULTI-CLUSTER FLOW



EXAMPLE ARGOCD APPLICATION

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: multi-cluster-app
spec:
  destination:
    server: https://kubernetes.default.svc
    namespace: app
  source:
    repoURL: https://github.com/veriqta/app
    targetRevision: main
    path: k8s
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

COMMON CHALLENGES

-  Cross-cluster networking complexity
-  Data consistency and replication
-  Workload placement and scheduling
-  Managing multiple cluster lifecycles
-  Security and access control
-  Observability across clusters
-  Disaster recovery and failover testing
-  Increased operational overhead
-  Cost management across regions

BEST PRACTICES

-  Define clear cluster boundaries
-  Use GitOps for all clusters
-  Implement multi-cluster monitoring
-  Use consistent tooling and standards
-  Automate cluster provisioning and updates
-  Implement strong security and RBAC
-  Test failover and DR regularly
-  Optimize workload placement and costs
-  Use service mesh for cross-cluster communication

REAL WORLD BENEFITS

-  Higher availability and resilience
-  Better performance and user experience
-  Separation of workloads and risk
-  Easier compliance with data regulations
-  Faster and safer deployments with GitOps
-  Improved disaster recovery capabilities
-  Optimized resource usage and costs
-  Scalable platform for global workloads
-  More secure and manageable infrastructure

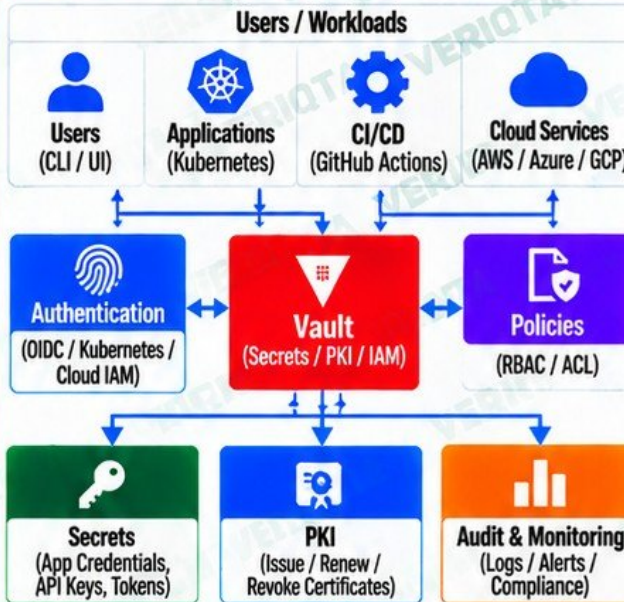
BUILD A PRODUCTION SECRETS AND PKI PLATFORM

Vault, certificate lifecycle, workload identity, and credential rotation.

KEY COMPONENTS

- Secrets Management** (HashiCorp Vault)
- PKI / Certificate Authority** (Internal PKI)
- Workload Identity** (IAM / OIDC / SPIFFE)
- Credential Rotation** (Automated)
- Policy & Access Control** (RBAC / ACL / Policies)
- Audit & Monitoring** (Logs / Alerts)

ARCHITECTURE OVERVIEW



PKI LIFECYCLE

- 1. CREATE CA** (Root / Intermediate)
- 2. ISSUE CERTIFICATE** (Users / Services / Workloads)
- 3. RENEW** (Automated Renewal)
- 4. REVOKE** (Compromised / Not Used)
- 5. DISTRIBUTE** (Workloads / Ingress / Service Mesh)
- 6. MONITOR** (Expiry / Usage / Audit)

WORKLOAD IDENTITY

- Kubernetes Service Accounts** (OIDC / Bound Tokens)
- Cloud Workload Identity** (AWS IAM / Azure Workload ID / GCP)
- SPIFFE / SPIRE** (Workload Identity Federation)
- Short-lived Credentials** (No long-lived secrets)
- Fine-grained Access** (Least Privilege)

SECRETS TYPES

- Database Credentials** (Username / Password)
- API Keys** (Third-party services)
- TLS Certificates** (Internal / External)
- Cloud Credentials** (AWS / Azure / GCP)
- Application Secrets** (Tokens / Encryption Keys)
- Configuration Data** (Environment / Feature Flags)

CREDENTIAL ROTATION

- Automated Rotation** (Scheduled / Event-based)
- Short-lived Credentials** (Dynamic secrets)
- Zero-Downtime Rotation** (Seamless reload)
- Validate After Rotation** (Health checks)
- Audit All Rotations** (Logs / Alerts)

EXAMPLE WORKFLOW



EXAMPLE VAULT COMMANDS

```
> # Enable PKI secrets engine
vault secrets enable pki
# Generate a CA certificate
vault write pki/root/generate/internal \
  common_name="veriqta-internal-ca" \
  ttl=8760h
# Issue a certificate
vault write pki/issue/app-role \
  common_name="app.veriqta.local" \
  ttl=720h
# Read a secret
vault kv get secret/app/db
```

COMMON CHALLENGES

- Exposed secrets in code or images
- Long-lived credentials
- Expired or untrusted certificates
- Complex certificate management
- Insufficient access controls
- Lack of monitoring and audit logs
- Rotation causing service downtime
- Workload identity misconfiguration

BEST PRACTICES

- Use short-lived and dynamic secrets
- Implement workload identity (OIDC/SPIFFE)
- Automate certificate issuance and renewal
- Enforce least privilege access (RBAC/Policies)
- Monitor certificate expiry and usage
- Enable audit logging and alerting
- Use zero-downtime rotation strategies
- Regularly test certificate revocation

REAL WORLD BENEFITS

- Higher security and reduced risk
- No hardcoded secrets
- Automated and centralized certificate management
- Improved compliance and auditability
- Simpler and safer access for workloads
- Faster incident response (revoke/rotate)
- Supports multi-cloud and Kubernetes
- Scalable and production-ready platform

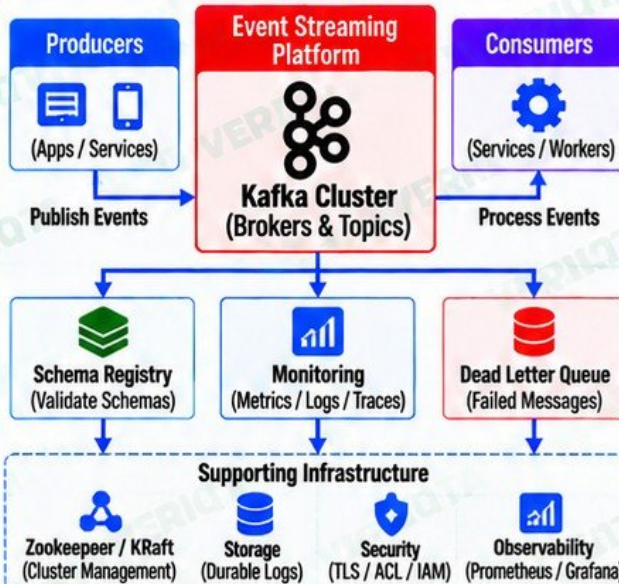
DESIGN A RESILIENT EVENT-DRIVEN INFRASTRUCTURE

Kafka, queue backpressure, retries, idempotency, and failure isolation.

KEY COMPONENTS

- Apache Kafka** (Event Streaming)
- Message Queues** (SQS / RabbitMQ)
- Schema Registry** (Schema Management)
- Consumers & Producers** (Applications / Services)
- Reliability Patterns** (Retries / Idempotency / DLQ)
- Monitoring & Observability** (Metrics / Logs / Traces)

ARCHITECTURE OVERVIEW



EVENT FLOW

- PRODUCE**
Create and publish events
- VALIDATE**
Check schema (Schema Registry)
- STORE**
Persist in Kafka topics
- CONSUME**
Process with consumer groups
- HANDLE FAILURES**
Retry / DLQ / Isolation
- MONITOR**
Track metrics and alerts

BACKPRESSURE AND FLOW CONTROL



RETRY AND IDEMPOTENCY

- Retry with Exponential Backoff** (Handle transient failures)
- Idempotent Processing** (Use unique event IDs)
- Deduplication** (Prevent duplicate side effects)
- Exactly/At-Least-Once Semantics** (Design for reliability)

FAILURE ISOLATION

- Isolate by Service/Topic** (Separate critical workloads)
- Bulkheads** (Limit blast radius)
- Dead Letter Queues** (Handle poison messages)
- Circuit Breakers** (Stop cascading failures)
- Multi-Region / Multi-AZ** (High availability and resilience)

EXAMPLE KAFKA SETUP

```
# Create a topic with replication and retention
kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --partitions 6 \
  --replication-factor 3 \
  --config retention.ms=604800000

# Consume messages
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning
```

TOOLS & TECHNOLOGIES

- Apache Kafka** (Event Streaming)
- RabbitMQ** (Message Queue)
- Apache Pulsar** (Alternative)
- Schema Registry** (Confluent / Apicurio)
- Prometheus + Grafana** (Monitoring & Alerting)

COMMON USE CASES

- E-commerce** (Orders, Payments)
- Financial Services** (Transactions, Fraud Detection)
- Real-Time Analytics** (Events, Clickstream)
- Microservices** (Event-Driven Communication)
- Audit Logs** (Compliance, Security)
- IoT / Device Data** (High Throughput Events)

COMMON CHALLENGES

- Consumer lag and queue buildup
- Handling burst traffic and backpressure
- Duplicate message processing
- Poison messages and failed consumers
- Schema evolution and compatibility
- Lack of monitoring and alerting
- Cascading failures across services
- Scaling consumers efficiently

BEST PRACTICES

- Use proper partitioning and replication
- Implement backpressure and rate limiting
- Design idempotent consumers
- Use retries with exponential backoff
- Isolate failures with DLQs and circuit breakers
- Monitor consumer lag and queue depth
- Automate scaling based on metrics
- Test failure scenarios regularly

REAL WORLD BENEFITS

- Highly resilient and scalable infrastructure
- Handles high traffic and burst loads
- Reduced data loss and improved reliability
- Faster recovery from failures
- Better isolation between services
- Improved system stability and performance
- Supports real-time analytics and decision making
- Scalable for global and multi-region deployments

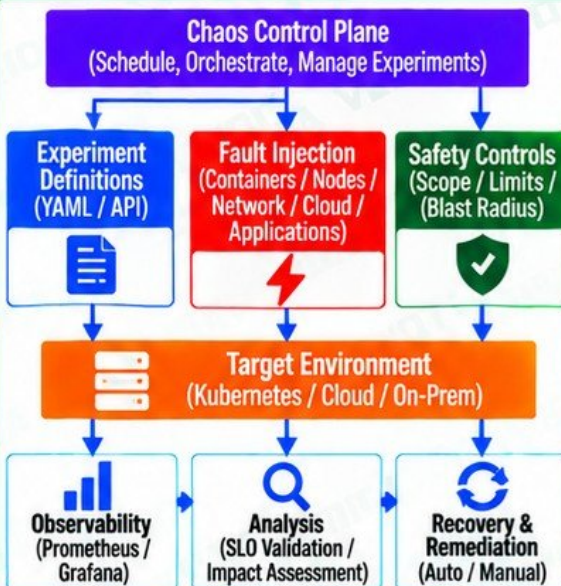
BUILD AN INFRASTRUCTURE CHAOS ENGINEERING PLATFORM

Fault injection, steady-state hypotheses, blast-radius controls, and recovery.

KEY COMPONENTS

-  **Chaos Experimentation Engine**
(Design, Schedule, Execute)
-  **Fault Injection Tools**
(Containers, Nodes, Network, Cloud, Applications)
-  **Hypothesis & Steady-State Validation**
(SLOs, Metrics, Alerts)
-  **Blast-Radius Controls**
(Scope, Safety, Guardrails)
-  **Observability & Analysis**
(Metrics, Logs, Traces)
-  **Reporting & Governance**
(Results, Lessons, Compliance)

ARCHITECTURE OVERVIEW



CHAOS EXPERIMENT TYPES

-  **Infrastructure Failures**
(Node, VM, AZ, Region loss)
-  **Container & Kubernetes**
(Pod kill, resource pressure, API failures)
-  **Network Chaos**
(Latency, packet loss, DNS failures, partition)
-  **Application Chaos**
(Service failure, dependency failure, error injection)
-  **Cloud Service Chaos**
(AWS / Azure / GCP API failures, service limits)
-  **Data Layer Chaos**
(Database failure, replica loss, storage latency)

EXAMPLE EXPERIMENTS

-  Kill random pods (Kubernetes)
-  Terminate a node (simulate outage)
-  Introduce network latency and packet loss
-  Throttle database connections
-  Simulate cloud service API failure
-  Increase CPU and memory pressure
-  Inject application errors (5xx)
-  Simulate DNS failure

BLAST-RADIUS CONTROLS

-  Define scope and boundaries (namespace, cluster, region)
-  Use canary environments during initial tests
-  Limit experiment duration and intensity
-  Exclude critical systems
-  Pre-checks and automated safety validations
-  Automatic rollback on breach (SLO / error budget)
-  Run experiments during maintenance windows

STEADY-STATE HYPOTHESES

-  Define expected behavior (e.g., service remains available)
-  Set SLOs and success criteria (latency, error rate, throughput)
-  Validate during and after experiment
-  Compare against steady-state metrics
-  Identify gaps and weaknesses
-  Update hypotheses based on results
-  Continuously improve resilience

TOOLS & TECHNOLOGIES

-  **Chaos Mesh** (Kubernetes)
-  **Gremlin** (Multi-Cloud)
-  **Litmus** (Kubernetes)
-  **Chaos Toolkit** (Multi-Platform)
-  **Prometheus** (Monitoring)
-  **Grafana** (Visualization)

EXAMPLE CHAOS MESH EXPERIMENT (POD FAILURE)

```

apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: pod-kill-experiment
spec:
  action: pod-kill
  mode: random-max-percent
  value: "30"
  selector:
    namespaces:
      - default
    labelSelectors:
      app: my-app
  duration: "60s"
  
```

COMMON CHALLENGES

- ✗ Uncontrolled blast radius
- ✗ Breaking critical services
- ✗ Inaccurate hypotheses
- ✗ Lack of observability
- ✗ Insufficient safety controls
- ✗ Inconsistent results
- ✗ Team resistance
- ✗ No action on findings

BEST PRACTICES

- ✓ Start small and increase complexity
- ✓ Define clear hypotheses and SLOs
- ✓ Use blast-radius controls and safeguards
- ✓ Automate rollback and recovery
- ✓ Run in non-production first
- ✓ Monitor everything (metrics, logs, traces)
- ✓ Document results and lessons learned
- ✓ Integrate with incident response processes

REAL WORLD BENEFITS

- ✓ Improved system resilience and reliability
- ✓ Faster recovery from real incidents
- ✓ Validated disaster recovery strategies
- ✓ Reduced mean time to recovery (MTTR)
- ✓ Identification of architectural weaknesses
- ✓ Greater confidence in system behavior
- ✓ Stronger incident response readiness
- ✓ Supports a culture of continuous improvement



@veriqta



VERIQTA



VERIQTA



VERIQTA



VERIQTA



@veriqta

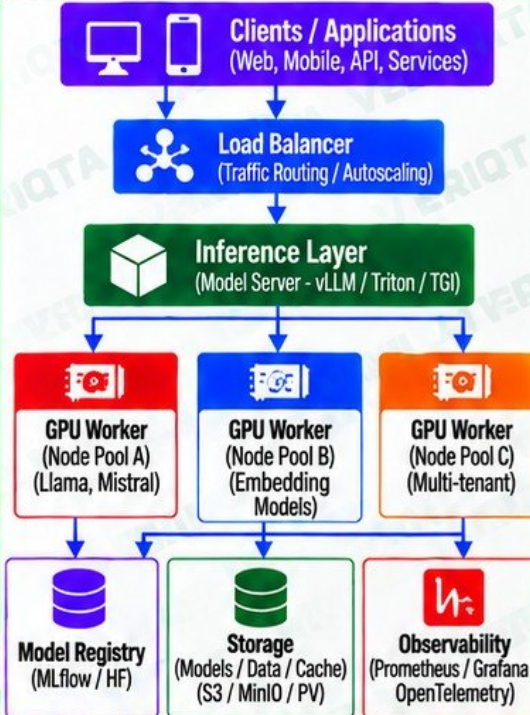
ENGINEER AN AI INFRASTRUCTURE OPERATIONS PLATFORM

GPU scheduling, inference serving, model observability, and resource isolation.

KEY COMPONENTS

- GPU Infrastructure**
(NVIDIA GPUs, Drivers, CUDA, Container Runtime)
- Workload Orchestration**
(Kubernetes, GPU Scheduling, Node Pools)
- Inference Serving**
(Model Deployment, Auto Scaling, Traffic Routing)
- Model Observability**
(Metrics, Traces, Logs, Drift Detection)
- Resource Isolation**
(Multi-tenancy, Quotas, Security, Network Policies)
- Storage & Model Management**
(Model Registry, Artifact Storage, Caching)

ARCHITECTURE OVERVIEW



GPU SCHEDULING STRATEGIES

- Kubernetes GPU Scheduling**
(Node Selectors, Taints/Tolerations)
- Resource Requests & Limits**
(nvidia.com/gpu, CPU, Memory)
- Node Pools**
(Separate by model type / team)
- GPU Sharing**
(MIG, MPS, vGPU)
- Queue & Priority Scheduling**
(Priority Classes, Fair Sharing)
- Autoscaling**
(Scale up/down based on demand)

INFERENCE SERVING OPTIONS

- vLLM**
(High performance, LLM serving)
- NVIDIA Triton Inference Server**
(Multi-model, production grade)
- Text Generation Inference (TGI)**
(Hugging Face, open source)
- Model Routing**
(A/B testing, canary, shadow)
- Batch and Streaming Inference**
(REST, gRPC, WebSocket)

MODEL OBSERVABILITY

- Metrics**
(latency, throughput, token/sec, GPU usage)
- Logs**
(requests, errors, system logs)
- Traces**
(end-to-end request tracing)
- Model Performance**
(accuracy, drift, bias, hallucination rate)
- Alerts**
(slo violations, GPU saturation, failures)
- Dashboards**
(Grafana, Prometheus, OpenTelemetry)

RESOURCE ISOLATION & MULTI-TENANCY

- Namespaces**
(Isolate teams / workloads)
- Quota & Limit Ranges**
(GPU, CPU, Memory, Storage)
- Network Policies**
(Control traffic between services)
- Pod Security Standards**
(Restrict privileges)
- Secrets Management**
(API keys, model creds, tokens)
- Dedicated Node Pools**
(By tenant / model type)

TOOLS & TECHNOLOGIES

- Kubernetes**
(Orchestration)
- NVIDIA GPU / CUDA**
- vLLM**
(LLM Serving)
- Triton**
(Inference Server)
- Hugging Face**
(TGI / Models)
- MLflow**
(Model Registry)
- Prometheus**
(Monitoring)
- Grafana**
(Visualization)
- OpenTelemetry**
(Traces / Metrics)
- MinIO**
(Model Storage)

COMMON CHALLENGES

- GPU resource contention
- Model deployment failures
- High inference latency
- Scaling under heavy load
- Model drift and quality degradation
- Noisy neighbor (multi-tenant issues)
- Insufficient observability
- Driver and CUDA compatibility issues
- Cost management (GPU spend)
- Security and data isolation

EXAMPLE KUBERNETES GPU DEPLOYMENT

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: llm-inference
spec:
  replicas: 2
  selector:
    matchLabels:
      app: llm-inference
  template:
    metadata:
      labels:
        app: llm-inference
    spec:
      containers:
        - name: vllm
          image: vllm/vllm:latest
          resources:
            limits:
              nvidia.com/gpu: 1
              cpu: "4"
              memory: 16Gi
  
```

BEST PRACTICES

- Use dedicated GPU node pools
- Set appropriate resource requests and limits
- Implement autoscaling based on real metrics
- Use model caching and warmup
- Monitor GPU, latency, and error rates
- Implement CI/CD for model deployments
- Use multi-tenant isolation (namespaces, quotas)
- Enable tracing and structured logging
- Implement canary and A/B testing
- Set SLOs and alerts
- Regularly test scalability and failure scenarios
- Optimize cost with right instance types and scheduling

REAL WORLD BENEFITS

- Scalable and reliable AI model serving
- Efficient GPU utilization and lower costs
- Improved model performance and observability
- Secure multi-tenant AI infrastructure
- Faster deployment and experimentation
- Better resource management and isolation

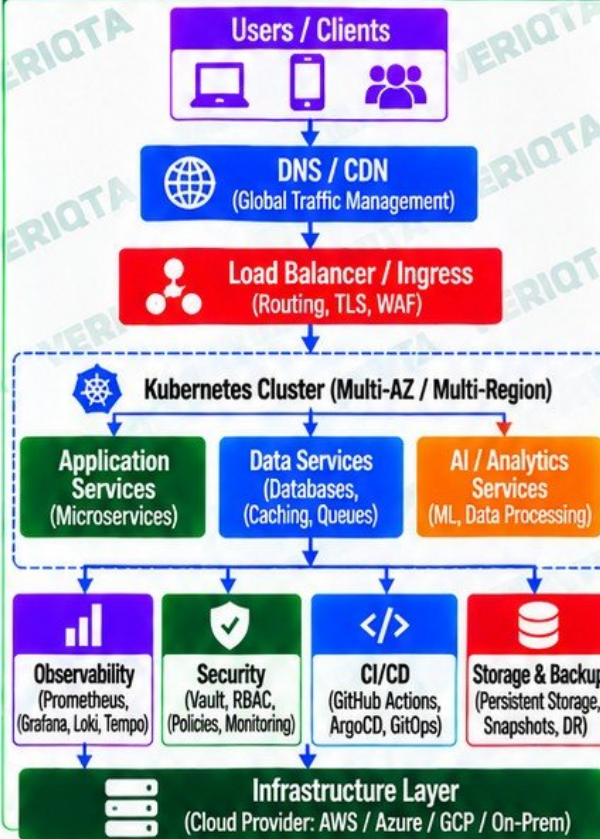
CAPSTONE: BUILD AND OPERATE AN ENTERPRISE INFRASTRUCTURE PLATFORM

Integrate security, networking, Kubernetes, CI/CD, observability, resilience, and incident response.

PLATFORM COMPONENTS

- Cloud / Hybrid Infrastructure**
(Compute, Storage, Networking)
- Kubernetes Platform**
(Clusters, Workloads, Ingress)
- CI/CD Pipeline**
(Build, Test, Deploy, GitOps)
- Security & Compliance**
(IAM, Secrets, Policies, Audit)
- Data Layer**
(Databases, Caching, Message Queues)
- Observability**
(Metrics, Logs, Traces, Alerts)
- Resilience & Disaster Recovery**
(Backups, Multi-Region, Failover)
- Incident Response**
(Detection, Triage, Remediation)

ARCHITECTURE OVERVIEW



END-TO-END FLOW

- 1. USER REQUEST**
Access application (Internet / Private Network)
- 2. DNS & CDN**
Resolve and route traffic
- 3. LOAD BALANCER / INGRESS**
Route to Kubernetes services
- 4. APPLICATION SERVICES**
Run in Kubernetes cluster
- 5. DATA SERVICES**
Persist and process data
- 6. CI/CD & GITOPS**
Deploy and manage changes
- 7. SECURITY CONTROLS**
Enforce policies and monitor
- 8. OBSERVABILITY**
Collect metrics, logs, traces
- 9. RESILIENCE & DR**
Backup, replicate, failover
- 10. INCIDENT RESPONSE**
Detect, triage and remediate

SECURITY INTEGRATION

- Identity & Access Management**
(RBAC, SSO, Least Privilege)
- Secrets Management**
(HashiCorp Vault / Cloud Secrets)
- Network Security**
(VPC, Security Groups, mTLS)
- Policy as Code**
(OPA, Kyverno, Pod Security)
- Continuous Compliance**
(Audit Logs, Monitoring, Alerts)

CI/CD & GITOPS

- Source Control**
(GitHub / GitLab)
- Automated Pipeline**
(Build, Test, Security Scan)
- Artifact Registry**
(Images, Helm Charts)
- GitOps Deployment**
(ArgoCD / Flux)
- Environment Promotion**
(Dev → Staging → Production)

OBSERVABILITY STACK

- Metrics**
(Prometheus + Grafana)
- Logs**
(Loki / ELK / OpenSearch)
- Traces**
(Tempo / Jaeger)
- Alerting**
(Alertmanager / PagerDuty)
- Dashboards**
(Real-time visibility)

RESILIENCE & DISASTER RECOVERY

- Automated Backups**
(Snapshots / Cross-Region)
- Multi-Region Deployment**
(Active-Active / Active-Passive)
- Disaster Recovery Plan**
(RTO / RPO)
- Chaos Engineering**
(Test failure scenarios)
- Regular Recovery Testing**
(Validated backups and failover)

INCIDENT RESPONSE

- Monitoring & Detection**
(Alerts / Anomaly Detection)
- Triage & Investigation**
(Logs / Metrics / Traces)
- Runbooks**
(Standardized Procedures)
- Controlled Remediation**
(Automated / Manual)
- Post-Incident Review**
(Root Cause Analysis / Action Items)

TOOLS & TECHNOLOGIES

- Kubernetes**
- Terraform**
- ArgoCD**
- GitHub Actions**
- Prometheus**
- Grafana**
- Loki**
- Vault**
- AWS**
- Azure**
- GCP**
- Helm**

COMMON CHALLENGES

- ✗ Complex multi-service integration
- ✗ Misconfigured networking and DNS
- ✗ Security misconfigurations
- ✗ CI/CD pipeline failures
- ✗ Insufficient observability
- ✗ Handling traffic spikes and failures
- ✗ Backup and recovery validation
- ✗ Incident response process gaps
- ✗ Managing costs at scale
- ✗ Cross-team coordination

REAL WORLD BENEFITS

- ✓ Scalable and secure platform
- ✓ Faster and safer deployments
- ✓ High availability and resilience
- ✓ Improved system observability
- ✓ Reduced downtime and faster incident recovery
- ✓ Automated and standardized operations
- ✓ Better security and compliance
- ✓ Cost optimization
- ✓ Supports global and multi-region deployments